

Unit – V

Intermediate SQL (Indexes and User Privileges)

Part 1: SQL Index

What You Will Learn

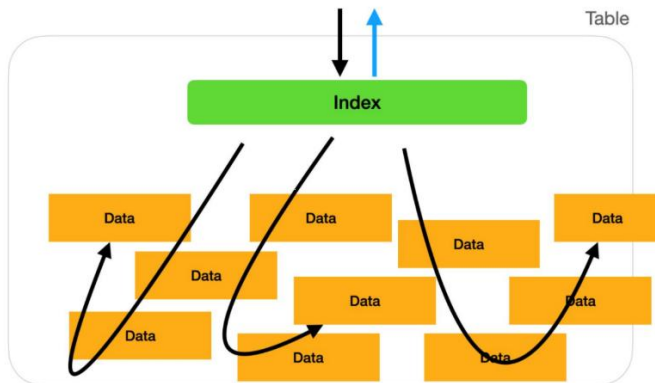
1. What an Index is and why it is essential for database performance.
2. How to create, manage, and apply indexes with different constraints.
3. The six core principles for using indexes effectively.

Understanding Indexes

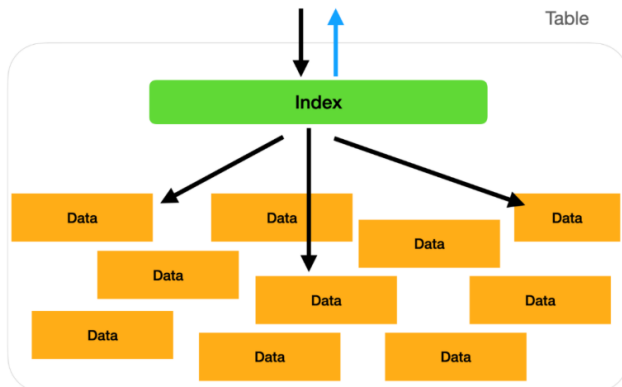
Concept & Benefits

The core concept of index technology is to mark specific columns in a table, extract their values, and store them in an organized manner separately from the main table data. When an SQL query executes, the database engine can directly search based on these indexes rather than scanning the entire table (Full Table Scan).

- **Without an Index:** The database engine scans through every record row by row to locate matching results, taking redundant query paths.



- **With an Index:** The database checks the index layer, finds the relevant pointers, and directly fetches the required data without taking detours.



Performance Note: Indexes show significant performance improvements when a single table exceeds **100,000 records** or when complex transactions/functions execute large volumes of SQL queries.

Trade-offs & Drawbacks

Indexes are not a one-size-fits-all solution.

- Every time data is inserted, updated, or deleted (INSERT, UPDATE, DELETE), the database must update the corresponding index structures.
- High-frequency write operations on indexed tables can degrade insertion speed. Always evaluate the trade-off to avoid scenarios where a query becomes 1 second faster, but writing new data becomes 2 seconds slower.

Index Syntax

Adding an Index

Adding an index is similar to adding foreign key constraints:

```
ALTER TABLE `new_schema`.`users`  
ADD INDEX `name_index` (`name`);
```

Alternatively, you can use the CREATE INDEX statement:

```
CREATE INDEX `name_index` ON `new_schema`.`users` (`name`);
```

Deleting an Index

To drop an index, specify the index name using DROP INDEX:

```
ALTER TABLE `new_schema`.`users`  
DROP INDEX `name_index`;
```

Index Categories

If no category is specified in MySQL, it defaults to standard INDEX. Below are three common specialized index categories:

1. PRIMARY KEY

Setting a Primary Key automatically creates a primary index.

- **Usage:** Primarily used on unique record identifiers (id).
- **Performance Benefit:** Extremely fast when joining tables (JOIN operations connecting id columns), providing peak lookup speeds.

2. UNIQUE INDEX

Enforces uniqueness on non-primary key columns while indexing them.

- Prevents duplicate entries in the specified column.
- Improves index evaluation efficiency.
- Note: PRIMARY KEY is a specialized form of UNIQUE INDEX.

3. FULLTEXT

- Specifically designed for text keyword searches (e.g., building search engines).
- Requires checking language support based on the specific database engine used.

Principles for Indexing Best Practices

1. **Prefer Uniqueness:** Try to choose UNIQUE INDEX wherever possible; otherwise, use standard INDEX/KEY.
2. **Target Complex Operations:** Add indexes to columns frequently referenced by complex grouping operations like GROUP BY.
3. **Index Query Hotspots with High Variance:** Focus on columns heavily used in WHERE clauses that contain high data cardinality (distinct values). For example, indexing identity_number is far more effective than indexing country_of_birth.
4. **Monitor Disk Space:** Indexes occupy additional disk storage; keep track of total index footprint.
5. **Index Small Value Columns:** Index smaller data fields over large ones (e.g., indexing an article's title is far more efficient than indexing the article's full content).
6. **Avoid Nullable Columns:** Try to avoid index on columns with NULL, which may affect the query efficiency.

Part 2: SQL User Privilege Management

What You Will Learn

1. How to create and manage database user accounts in SQL.
2. How to grant and manage user permissions based on network context.

Creating a User

To create a new database user account:

```
CREATE USER 'john'@'localhost' IDENTIFIED BY 'password';
```

Syntax Breakdown:

1. CREATE USER: The SQL DDL command for account creation.
2. 'john'@'localhost':
 - 'john': User account username.
 - 'localhost': Network context restricting where the user can connect from.
3. IDENTIFIED BY 'password': Defines the authentication password for the user.

Network Contexts

The network context restricts the host location from which a user is permitted to authenticate:

Host Context	Description	Security Level
localhost	User can only connect from the same server machine hosting the database.	High
110.78.9.12	User is strictly restricted to connecting from a specific IP address.	Medium / High
%	Wildcard: User can connect from any remote location or IP address.	Low (Use with care)

Viewing Users

To inspect existing users and their global privileges:

```
SELECT * FROM `mysql`.`user`;
```

Example Output:

Host	User	Select_priv	Insert_priv
localhost	root	Y	Y
localhost	john	N	N

Granting Privileges

After creating a user, permissions must be explicitly granted:

```
-- Grant all privileges on a specific table to john
```

```
GRANT ALL ON `new_schema`.`orders` TO 'john'@'localhost';
```

```
-- Grant SELECT privilege on all tables in new_schema to root from a specific IP
```

```
GRANT SELECT ON `new_schema`.* TO 'root'@'150.10.12.1';
```

Syntax Breakdown:

- GRANT [privilege] ON: Assigns explicit rights (e.g., ALL, SELECT, INSERT, UPDATE).
- [schema].[table]: Specifies target schema and table scope (* targets all tables).
- TO '[user]'@[host]': Specifies the recipient user account.

Example:

1. Product Price at a Given Date

Table: Products

Column Name	Type
product_id	int
new_price	int
change_date	date

(product_id, change_date) is the primary key (combination of columns with unique values) of this table.

Problem Summary

Each row of this table indicates that the price of some product was changed to a new price at some date.

Initially, all products have price 10.

Write a solution to find the prices of all products on the date 2019-08-16.

SQL Schema

Create table If Not Exists Products (product_id int, new_price int, change_date date)

Truncate table Products

insert into Products (product_id, new_price, change_date) values ('1', '20', '2019-08-14')

insert into Products (product_id, new_price, change_date) values ('2', '50', '2019-08-14')

insert into Products (product_id, new_price, change_date) values ('1', '30', '2019-08-15')

insert into Products (product_id, new_price, change_date) values ('1', '35', '2019-08-16')

insert into Products (product_id, new_price, change_date) values ('2', '65', '2019-08-17')

insert into Products (product_id, new_price, change_date) values ('3', '20', '2019-08-18')

Products

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

MySQL Solution

</> Code

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH LatestPrice AS (
3      SELECT
4          product_id,
5          new_price,
6          ROW_NUMBER() OVER (
7              PARTITION BY product_id
8              ORDER BY change_date DESC
9          ) AS rn
10     FROM Products
11     WHERE change_date <= '2019-08-16'
12 )
13
14 SELECT
15     p.product_id,
16     COALESCE(lp.new_price, 10) AS price
17 FROM
18     (SELECT DISTINCT product_id FROM Products) p
19 LEFT JOIN LatestPrice lp
20     ON p.product_id = lp.product_id
21     AND lp.rn = 1;
```

 Testcase |  Test Result

Output

product_id	price
1	35
2	50
3	10

2. Product Sales Analysis I

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

sale_id,year is the primary key column for this table.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key column for this table.

Problem Summary

Write a solution to report the product_name, year, and price for each sale_id in the Sales table.

SQL Schema

Create table Sales (sale_id int, product_id int, year int, quantity int, price int);

Create table Product (product_id int, product_name varchar(10));

Truncate table Sales;

insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');

insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');

Truncate table Product;

insert into Product (product_id, product_name) values ('100', 'Nokia');

insert into Product (product_id, product_name) values ('200', 'Apple');

insert into Product (product_id, product_name) values ('300', 'Samsung');

Sales

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product

product_id	product_name
100	Nokia
200	Apple
300	Samsung

MySQL Solution

☒ Testcase ☒ Test Result

Output

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```